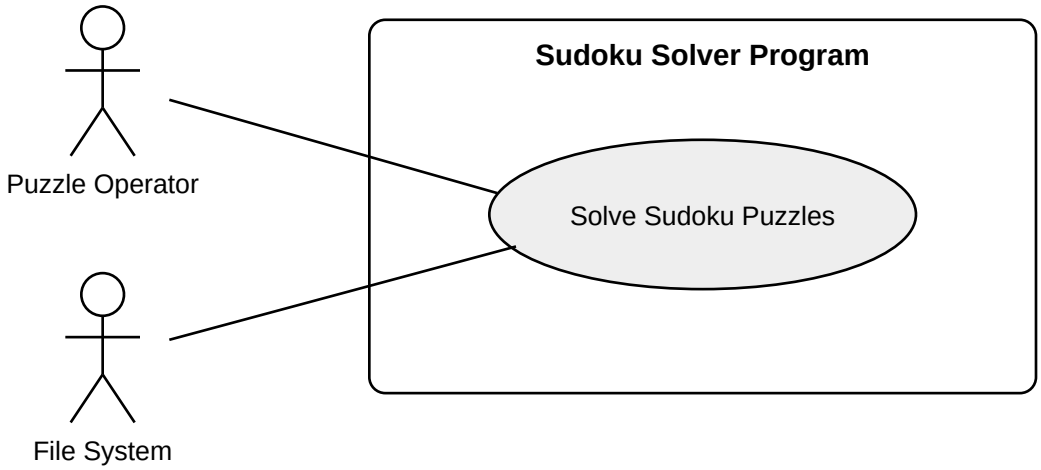
Use Case: Solve Sudoku Puzzles

Sudoku Solver Program (Assignment 4)

a. Identify/discover a use case

- **Use case:** Solve Sudoku Puzzles
- **Primary actor:** Puzzle Operator — the person who runs the program.
- **Supporting actor:** File System — provides the puzzle files to the program and stores the solution files.
- **Goal:** The operator wants a solution file for each supplied Sudoku puzzle, containing the solutions or a “No solution found” message.
- **Use case diagram:**



b. Brief description

This use case allows a Puzzle Operator to run the program, which reads up to five puzzle files, checks and solves each one, and writes each puzzle's file name, original grid, and solution(s) (or a message that no solution was found) to a matching `solutionN.txt` file.

c. Step-by-step flow (outline)

1. The Puzzle Operator starts the program.
2. The program selects the first puzzle file.
3. The program requests the puzzle file from the File System.
4. The File System provides the file. (If the file is missing, the program reports it on the console, creates no solution file, and moves to the next puzzle.)
5. The program creates `solutionN.txt` and writes the puzzle file name to it.
6. The program checks that the file contains a valid puzzle.
7. The program writes the original puzzle to the solution file.

8. The program checks the puzzle's starting clues for conflicts.
9. The program searches for solutions using depth-first search with backtracking.
10. The program writes each solution it finds to the solution file, or writes “No solution found.”
11. The program writes the number of solutions found to the solution file.
12. The program closes the solution file and reports on the console that it was written.
13. The program repeats steps 2–12 for the remaining puzzle files.
14. The program finishes after processing all five puzzle numbers.

d. Detailed description

Use case: Solve Sudoku Puzzles

Preconditions

- The Puzzle Operator can start the program.
- The expected puzzle files (`../puzzle1.txt` through `../puzzle5.txt`) are available in the locations the program uses.
- The program has permission to create files in the folder it runs from.

Basic flow

1. **Operator starts the program.** The Puzzle Operator runs the Sudoku Solver Program.
2. **Program selects a puzzle file.** The program selects the next file in its set of five puzzle numbers.
3. **Program reads the file.** The program requests the selected file from the File System and reads the puzzle's cells.
4. **Program creates the solution file.** The program creates `solutionN.txt` through the File System, where N is the puzzle number, and writes the puzzle file name at the top.
5. **Program validates the file format.** The program checks that the file has exactly 81 cells and that each cell is a digit from 1 through 9 or an underscore for an empty cell.
6. **Program writes the original puzzle.** If the file is valid, the program writes the puzzle grid to the solution file.
7. **Program checks the starting clues.** The program checks that the given numbers do not conflict with Sudoku rules.
8. **Program searches for solutions.** If the clues are valid, the program uses recursive depth-first search with backtracking to find every valid completion. It always fills the empty cell with the fewest candidates first.
9. **Program writes the result.** The program writes each solution it finds to the solution file, followed by the total number of solutions. If it finds none, it writes “No solution found.”
10. **Program closes the solution file.** The program closes `solutionN.txt` and prints a “Wrote solutionN.txt” confirmation on the console.
11. **Program processes the next file.** The program repeats the process for the next puzzle.
12. **Program completes the run.** After processing all five puzzle numbers, the program exits.

Alternative courses of action

- **A1. A puzzle file cannot be opened.** The program displays a console message identifying the file, creates no solution file for it, and continues with the next file. The solution files therefore always match the input files that exist.
- **A2. A puzzle file has missing, extra, or invalid cells.** The program writes a format error identifying the file into that puzzle's solution file, skips solving, and continues with the next file.
- **A3. The starting clues conflict.** The program writes “No solution found” to the solution file and continues.
- **A4. The puzzle has no valid completion.** The program writes “No solution found” to the solution file and continues.
- **A5. The puzzle has more than one valid completion.** The program writes every solution to the solution file, followed by the total, and continues.
- **A6. A solution file cannot be created.** The program displays a console error naming the file, skips that puzzle, and continues.

Postcondition

The program has attempted to process all five puzzle files. A `solutionN.txt` file exists for each puzzle file that could be opened, and each contains the results for that puzzle.